Automation through scripting has become a central practice in modern cloud operations, allowing infrastructure to be provisioned, configured and managed in a consistent, repeatable and efficient manner. The Bash script above demonstrates this principle in the context of Microsoft Azure, where a single script is used to create a resource group, provision a virtual machine, open the required network ports and execute a series of configuration commands once the instance is running. Rather than performing these steps manually through the Azure portal, the entire process is captured in code that can be executed on demand, version controlled and reused across projects.

One of the clearest benefits of this approach is consistency. When infrastructure is provisioned manually, there is a risk of human error, missed configuration steps or inconsistencies between environments. Wiggins (2017) addresses this issue directly in the Twelve Factor App methodology, which advocates for treating configuration and environment setup as code so that development, staging and production environments remain as similar as possible. Automation scripts embody this principle by ensuring that every deployment follows exactly the same sequence of steps, removing variation between one provisioning run and the next.

Automation also supports scalability, which is particularly important as cloud workloads grow in size and complexity. Mungoli (2023) discusses how distributed AI frameworks depend heavily on the ability to provision and scale compute resources quickly and reliably in response to changing workload demands. While the script above provisions a single virtual machine, the same logic can be extended and parameterised to provision multiple instances, or integrated into a larger orchestration pipeline, allowing infrastructure to scale up or down without manual intervention. This becomes increasingly valuable in environments where workloads are unpredictable or where rapid response to demand is required.

A further advantage of scripted automation lies in its compatibility with modern architectural approaches such as microservices. Oliveira and Hugo (2021) explain that event driven microservices architectures rely on infrastructure that can be provisioned and reconfigured quickly in response to events, rather than infrastructure that is static and manually maintained. Automation scripts support this model by allowing new services or instances to be spun up automatically as part of a broader event driven workflow, reducing the delay between a triggering event and the availability of the required resources.

Beyond consistency and scalability, automation reduces the operational burden on teams by removing repetitive manual tasks. Once a script has been written and tested, it can be executed repeatedly with minimal oversight, freeing team members to focus on higher value work such as architecture design or performance optimisation. It also improves auditability, since the script itself serves as documentation of exactly what was provisioned and how.

In summary, scripting and automation are fundamental to efficient cloud operations. They reduce human error, support scalable and event driven architectures, and align with established best practices for environment consistency, as reflected in the work of Wiggins (2017), Mungoli (2023) and Oliveira and Hugo (2021).

References

Mungoli, N. (2023) *Scalable, distributed AI frameworks: leveraging cloud computing for enhanced deep learning performance and efficiency*.

Oliveira, R. and Hugo, F. (2021) *Practical event driven microservices architecture*. New York: Apress.

Wiggins, A. (2017) *The twelve factor app*. Available at: https://12factor.net (Accessed: 23 August 2026).

Screenshot of batch script below.

bubble_sort.py | Release Notes: 1.128.0 | index.ts 5 | # Variables Untitled-1

```bash
# Variables
RESOURCE_GROUP="DevTeamProject-RG"
LOCATION="uksouth"
VM_NAME="devteam-vm01"
VM_IMAGE="Ubuntu2204"
ADMIN_USERNAME="azureadmin"
VM_SIZE="Standard_B1s"

# Step 1: Log in to Azure (assumes interactive login or service principal already configured)
echo "Logging in to Azure..."
az login

# Step 2: Create a resource group
echo "Creating resource group: $RESOURCE_GROUP"
az group create --name "$RESOURCE_GROUP" --location "$LOCATION"

# Step 3: Create the virtual machine
echo "Creating virtual machine: $VM_NAME"
az vm create \
  --resource-group "$RESOURCE_GROUP" \
  --name "$VM_NAME" \
  --image "$VM_IMAGE" \
  --admin-username "$ADMIN_USERNAME" \
  --generate-ssh-keys \
  --size "$VM_SIZE"

# Step 4: Open port 22 for SSH access
echo "Opening port 22 for SSH access"
az vm open-port --port 22 --resource-group "$RESOURCE_GROUP" --name "$VM_NAME"

# Step 5: Retrieve the public IP address of the VM
PUBLIC_IP=$(az vm show -d --resource-group "$RESOURCE_GROUP" --name "$VM_NAME" --query publicIps -o tsv)
echo "VM provisioned successfully. Public IP: $PUBLIC_IP"

# Step 6: Run basic configuration commands on the VM via SSH
echo "Running initial setup commands on the VM..."
ssh -o StrictHostKeyChecking=no "$ADMIN_USERNAME@$PUBLIC_IP" << 'EOF'
  sudo apt-get update -y
  sudo apt-get upgrade -y
  sudo apt-get install -y nginx
  sudo systemctl enable nginx
  sudo systemctl start nginx
  echo "Nginx installed and running."
EOF

echo "Provisioning and configuration complete."
```